Data Structures (ETCCDS202)

Unit–2 Assignment: Linear Data Structures

Name of the School: School of Engineering & Technology

Program : B.Tech (CSE / AIML)

Section : A

Student Name : Disha Saini

Roll No. : 2501730318

Submitted To : Ms. Neetu

1. Introduction

Linear data structures store elements in a sequential manner, where each element is connected to the next in a single line. These structures are fundamental in computer science and are widely used for organizing and managing data efficiently.

In this assignment, we implemented various linear data structures from scratch using Python. These include Dynamic Arrays, Linked Lists, Stack, and Queue. We also applied these concepts to solve a real-world problem: checking for balanced parentheses.

2. Dynamic Array Analysis

A dynamic array is an array that can automatically resize itself when it becomes full. Unlike static arrays, it does not have a fixed size.

When we use the append() operation:

- If space is available, the element is added directly - takes **O(1)** time.
- If the array is full, a new array with double capacity is created and all elements are copied, takes **O(n)** time.

However, resizing does not happen frequently. Therefore, over many operations, the average time taken per append is:

<u>Amortized Time Complexity = O(1)</u>

This makes dynamic arrays efficient for most use cases.

3. Singly Linked List

A singly linked list is a dynamic data structure made up of nodes, where each node contains data and a reference to the next node in the sequence.

A singly linked list consists of nodes where each node contains:

- Data
- A reference (pointer) to the next node

Unlike arrays, linked lists do not require contiguous memory, which makes insertion and deletion operations more flexible. In this implementation, we performed operations such as insertion at the beginning, insertion at the end, deletion by value, and traversal. Insertion at the beginning is efficient as it takes constant time, while insertion at the end and deletion require traversal of the list and therefore take linear time. The main advantage of a singly linked list is that it eliminates the need for shifting elements, which is required in arrays.

Key operations:

- Insertion at beginning → O(1)
- Insertion at end → O(n)
- Deletion by value → O(n)
- Traversal → O(n)

Linked lists are useful because they allow easy insertion and deletion without shifting elements.

4. Doubly Linked List

A doubly linked list is an extension of a singly linked list where each node contains two references: one to the next node and another to the previous node. This allows traversal in both forward and backward directions, making certain operations more efficient.

In this assignment, we implemented operations such as inserting a node after a given target and deleting a node at a specific position.

Additional operations:

- Insert after a given node
- Delete a node at a specific position

While doubly linked lists provide greater flexibility compared to singly linked lists, they require additional memory to store the extra pointer.

5. Stack (LIFO)

A stack is a linear data structure that follows the Last In First Out (LIFO) principle, meaning the most recently added element is the first one to be removed. In this assignment, the stack was implemented using a singly linked list to ensure efficient operations.

All stack operations, including push, pop, and peek, are performed at the head of the linked list, which allows them to run in constant time.

Operations:

- push() → O(1)
- pop() → O(1)
- peek() → O(1)

Stacks are commonly used in applications such as function calls, undo operations, and expression evaluation.

6. Queue (FIFO)

A queue is another linear data structure that follows the First In First Out (FIFO) principle, where the first element inserted is the first one to be removed. In this implementation, the queue was built using a singly linked list with both head and tail pointers. This design ensures that both enqueue and dequeue operations can be performed in constant time.

Operations and Time Complexity:

- enqueue() → O(1)
- dequeue() → O(1)
- front() → O(1)

To achieve constant time operations, both head and tail pointers are maintained.

Queues are widely used in scenarios such as task scheduling, buffering, and breadth-first search in graphs.

7. Results and Observations

The implemented data structures were tested using multiple test cases to verify their correctness and performance. The dynamic array successfully resized itself when the capacity was reached, and the resizing process was clearly visible in the output through printed messages. After inserting more than the initial capacity, the array doubled its size and continued to store elements correctly. The pop operation also worked as expected by removing elements from the end.

The singly linked list correctly performed insertion at both the beginning and end of the list. The deletion operation removed the specified value, and traversal displayed the updated list accurately after each operation. The doubly linked list allowed insertion after a given node and deletion at specific positions, while maintaining proper forward and backward links between nodes.

The stack implementation followed the Last In First Out principle correctly. The push, pop, and peek operations worked efficiently, and the correct element was always removed or accessed from the top of the stack. Similarly, the queue followed the First In First Out principle, where elements were enqueued at the rear and dequeued from the front without errors.

The balanced parentheses checker produced correct results for all test cases. It correctly identified valid expressions such as "([])" and rejected invalid ones like "([)]" and "((((". The empty string was also correctly considered as balanced.

Overall, all data structures behaved as expected, and the outputs matched the intended functionality, confirming the correctness of the implementation.

8. Conclusion

This assignment provided a comprehensive understanding of linear data structures and their practical implementation. It demonstrated how dynamic arrays handle resizing, how linked lists allow flexible memory usage, and how stacks and queues operate based on specific principles. Additionally, applying these concepts to solve a real problem helped reinforce their importance in programming. Overall, the assignment improved both conceptual understanding and practical coding skills related to data structures.

9. References

- Class notes from Data Structures
- Lecture PPTs
- GeeksforGeeks